

Unity Programmer Task - Heitor Ivaza Lopes

A base class, `ItemScriptableObject`, was created to define every item in the game, storing its ID, name, sprite, description, and base price. To support stackable items, a `QuantifiedItem` class was implemented, combining an `ItemScriptableObject` reference with its quantity. The inventory consists of an array of 20 `QuantifiedItem` slots, allowing direct placement and removal of items. When adding an item, the system checks whether it already exists in the inventory and increases its quantity; otherwise, it is placed in the first available slot.

The inventory UI uses a pool of 20 slots that updates dynamically through the `RenderInventory` function whenever the inventory changes. Players can inspect item information by hovering over or selecting a slot and can reorganize the inventory by dragging, swapping, or moving items between occupied and empty slots.

The save system was implemented using `JsonUtility` to serialize and deserialize inventory data. Since `ScriptableObject`s cannot be serialized directly, each inventory entry is converted into a `QuantifiedItemSaveData` object containing the item ID, quantity, and slot index. During loading, the item is retrieved from a centralized item database using its ID and restored to its original slot with the correct quantity, ensuring full slot-based persistence. The loading routine is executed automatically during game initialization, whereas save and data-clearing operations are triggered through dedicated actions available in the pause menu.

I believe my performance throughout the project was strong. I completed all required systems, maintained a clear development workflow through frequent commits, and reserved time for testing and polishing the final implementation.